Deep Learning: Attention, Transformer

Lecture 09.12

Foma Shipilov

Contents

1 Attention, Transformer **2**
1.1 Attention Layers . . . 2
1.1.1 Previous architectures . . . 2
1.1.2 Attention layer . . . 2
1.1.3 Self-attention: idea . . . 3
1.1.4 Self-attention . . . 3
1.1.5 Self-attention step by step . . . 3
1.1.6 Self-attention in matrix form . . . 4
1.1.7 Attention matrix . . . 4
1.1.8 Other score functions . . . 5
1.1.9 Complexity . . . 5
1.1.10 Multi-Head Attention: idea . . . 5
1.1.11 Multi-Head Attention . . . 6
1.1.12 Multi-Head Attention: step by step . . . 6
1.1.13 Multi-Head Attention: trainable parameters . . . 6
1.2 Transformer Model . . . 7
1.2.1 Transformer model overview . . . 7
1.2.2 Encoder–encoder attention . . . 7
1.2.3 Decoder–decoder attention . . . 7
1.2.4 Encoder–decoder attention . . . 8
1.2.5 Other operations . . . 8
1.2.6 Positional encoding . . . 8
1.2.7 Transformer size . . . 9
1.2.8 Training details . . . 9
1.2.9 Training tricks . . . 9
1.2.10 Output generation: greedy search . . . 9
1.2.11 Output generation: beam search . . . 10
1.2.12 BLEU metric for NMT . . . 10
1.3 Metrics and Inference for sequence generation . . . 10
1.4 Transformer–based LMs and contextualized word embeddings: ELMO, BERT, GPT . . . 10
1.5 Transformer for Computer Vision . . . 10

Language Models

Autoregressive Modelling

The joint probability of a sequence is factored into a product of conditionals:

$$p(x) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2)\dots p(x_T|x_1\dots x_{T-1})$$

Generation starts from ⟨BOS⟩ and proceeds step by step — at each step a token is sampled and fed back into the model:

$$\langle\text{BOS}\rangle \to \text{RNN} \to \text{Linear} \to \text{Logits} \in \mathbb{R}^{|v|} \to \text{softmax} \to p(x|x_0 = \langle\text{BOS}\rangle)$$
$$\text{sample} \quad x_1 \to \text{RNN} \to \text{Linear} \to \text{Logits} \to p(x|x_0 = \langle\text{BOS}\rangle, x_1 = x_1)$$
$$\text{sample} \quad x_2 \to \text{RNN} \to \text{Linear} \to \text{Logits} \to p(x|x_0 = \langle\text{BOS}\rangle, x_1 = x_1)$$

Until ⟨EOS⟩ — End of Sequence.

Training a Language Model

Let θ be the model parameters, $X = (x_1, \dots, x_T)$ a real sequence. Training minimizes the negative log-likelihood:

$$p_\theta(x) = p_\theta(x_0)p_\theta(x_1|x_0)\dots p(x_T|x_0, x_1, \dots, x_{T-1})$$
$$\|$$
$$1$$

$$-\sum_{t=1}^{T} \log p_\theta(x_t|x_0, \dots, x_{t-1}) \to \min$$

1 Attention, Transformer

1.1 Attention Layers

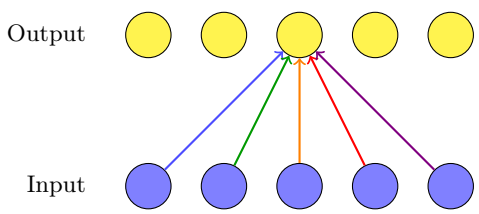
1.1.1 Previous architectures

Before attention mechanisms were introduced, three main types of layers were used to process sequences, each with significant drawbacks:

Fully-connected	**Convolutional**	**Recurrent**
Problems: • Fixed window • A lot of weights	*Problems:* • Local dependencies only	*Problems:* • Slow training • Troubles with gradients

1.1.2 Attention layer

Unlike previous architectures, the attention layer processes all tokens simultaneously. For each output, the model dynamically computes how relevant every input token is and takes a weighted sum — tokens that matter more get higher weights.



The attention layer addresses all the limitations of previous architectures:

- Look at all inputs to learn more global dependencies
- Share weights to work with variable length window
- Do not use autoregressive structure to make training parallelizable

1.1.3 Self-attention: idea

Each token embedding is updated using information from all other tokens in the sequence. So for each token, the model does two things:

- Find tokens relevant to the current one (compute relevance scores)
- Compute the weighted combination of their embeddings

Relevance scores are dot products of queries and keys, normalized via softmax:

$$p = \text{softmax}(k_1^T q_i,\ k_2^T q_i,\ \ldots,\ k_l^T q_i)$$

The updated embedding is a weighted sum of value vectors:

$$\tilde{x}_i = \sum_{j=1}^{L} p_j \cdot V_j$$

where q_i — query for token i (what this token is looking for), k_j — key for token j (what it has to offer), V_j — value for token j (what it actually contains).

1.1.4 Self-attention

Now we can write the same operation in matrix form for all tokens simultaneously. To compute relevance scores we use two linear transformations of the input:

$$Q = XW^Q, \quad K = XW^K$$

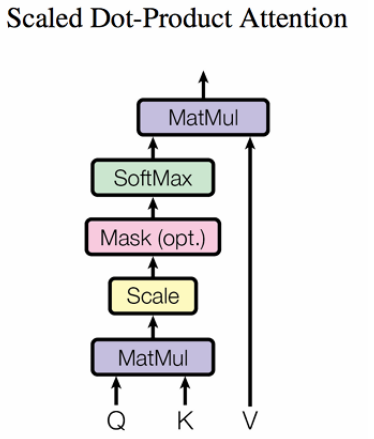
And to compute final embeddings we use the third one - V - values:

$$V = XW^V$$

where $Q, K, V \in \mathbb{R}^{L \times d}$. The attention output is then:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

$X \in \mathbb{R}^{L \times d} \to A = QK^T \in \mathbb{R}^{L \times L}$ — is the attention matrix (entry (i, j) reflects how relevant token j is to token i).
*Dividing by $\sqrt{d_k}$ prevents the dot products from growing too large before softmax.



1.1.5 Self-attention step by step

Step 1. Compute queries, keys, values for all inputs using the same linear transformations:

$$q_i = x_i W^Q, \qquad k_i = x_i W^K, \qquad v_i = x_i W^V$$

Step 2. For token i, compute relevance scores using query i and all the keys, then normalize via softmax:

Consider an example: We have two input tokens — "Thinking" and "Machines". Each embedding has 4 dimensions, each query/key/value vector has 4 dimensions, $d_k = 64$, so $\sqrt{d_k} = 8$.

Operation	Token 1	Token 2
Input	"Thinking"	"Machines"
Embedding x_i	x_1	x_2
Query $q_i = x_i W^Q$	q_1	q_2
Key $k_i = x_i W^K$	k_1	k_2
Value $v_i = x_i W^V$	v_1	v_2
Score $q_1 \cdot k_j^T$	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Scale $/\sqrt{d_k}$ $(= 8)$	$112/8 = 14$	$96/8 = 12$
Softmax	0.88	0.12
Weighted sum	$z_1 = 0.88 \cdot v_1 + 0.12 \cdot v_2$	

Token 1 attends mostly to itself — its weight 0.88 is significantly higher than the weight of token 2.

Step 3. Compute weighted combination of values, weights from relevance scores:

$$z_1 = 0.88 \cdot v_1 + 0.12 \cdot v_2$$

z_1 is the new embedding of token 1 — now it knows about all other tokens in the sequence.

1.1.6 Self-attention in matrix form

All operations from the 1.1.5 can be written compactly in matrix form. Q, K, V are obtained via linear projections of the input matrix X:

Step 1. Project input into Q, K, V:

$$X \times W^Q = Q, \qquad X \times W^K = K, \qquad X \times W^V = V$$

Each row of Q, K, V corresponds to one token.

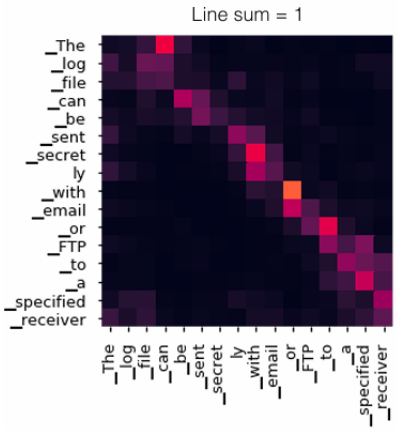
Step 2. Compute attention output Z:

$$Z = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

$$QK^T \xrightarrow{\text{gives}} \text{relevance scores for all token pairs} \xrightarrow{/\sqrt{d_k}} \text{scale} \xrightarrow{\text{softmax}} \text{weights} \xrightarrow{\times V} Z$$

1.1.7 Attention matrix

The result of softmax $\left(\frac{QK^T}{\sqrt{d_k}}\right)$ is an $L \times L$ matrix where entry (i, j) is the attention weight of token i towards token j. Each row sums to 1.



1.1.8 Other score functions

Name	Alignment score function	Citation
Content-base attention	$\text{score}(s_t, \boldsymbol{h}_i) = \text{cosine}[s_t, \boldsymbol{h}_i]$	Graves2014
Additive(*)	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \mathbf{v}_a^T \tanh(\mathbf{W}_a[\boldsymbol{s}_t; \boldsymbol{h}_i])$	Bahdanau2015
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a \boldsymbol{s}_t)$ *Note: This simplifies the softmax alignment to only depend on the target position.*	Luong2015
General	$\text{score}(s_t, \boldsymbol{h}_i) = s_t^T \mathbf{W}_a \boldsymbol{h}_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer	Luong2015
Dot-Product	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^T \boldsymbol{h}_i$	Luong2015
Scaled Dot-Product(∧)	$\text{score}(s_t, h_i) = \frac{s_i^T h_i}{\sqrt{n}}$ *Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.*	Vaswani2017

1.1.9 Complexity

$L \to n$

Table 1: Maximum path lengths, per–layer complexity and minimum number of sequential operations for different layer types, n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r — the size of the neighborhood in restricted self—attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Self-Attention is the most parallelizable and captures long-range dependencies in a single step. The downside is $O(n^2 \cdot d)$ complexity — the longer the sequence, the more expensive it gets. **Restricted self-attention** reduces complexity to $O(r \cdot n \cdot d)$ by limiting each token to only r neighbors, at the cost of losing global context.

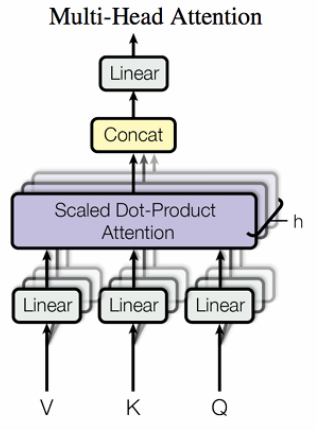
1.1.10 Multi-Head Attention: idea

A single attention head learns one type of relevance. But language has many different types of dependencies simultaneously — syntactic, semantic, coreference, and others.

That is why Multi-Head Attention was introduced — it runs h attention heads in parallel, each learning its own type of relevance. The outputs are then concatenated:

- **Single-head attention** — one relevance pattern for all tokens
- **Multi-head attention** — h different relevance patterns, each capturing a different type of relevance

1.1.11 Multi-Head Attention



Each head has its own projection matrices and learns its own type of relevance:

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

The outputs are concatenated and projected back:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^0$$

1.1.12 Multi-Head Attention: step by step

Step 1. Take the input sentence.

Step 2. We embed each word.*

Step 3. Split into 8 heads. We multiply X or R with weight matrices to get Q_i, K_i, V_i for each head.

Step 4. Calculate attention using the resulting $Q/K/V$ matrices.

Step 5. Concatenate the resulting Z matrices, then multiply with weight matrix W^0 to produce the output of the layer.

* In all encoders other than N0, we don't need embedding. We start directly with the output of the encoder right below this one

1.1.13 Multi-Head Attention: trainable parameters

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^0$$

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

Each head has its own set of projection matrices. Their dimensions are chosen so that the total computation stays comparable to single-head attention:

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

$$W^0 \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

where $d_k = d_v = d_{\text{model}}/h$ — each head works in a lower-dimensional space $d' = \frac{d}{h}$, but all h heads together cover the full dimensionality d_{model}.

h — num heads

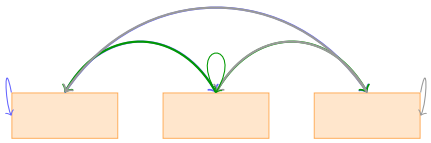
1.2 Transformer Model

1.2.1 Transformer model overview

The Transformer was originally proposed for neural machine translation — a seq2seq task where the input and output are sequences of different lengths.

- Encoder–decoder architecture
- 3 types of attention operators:
  - **Encoder–encoder** (self–attention):tokens of the input sequence attend to each other to build context-aware representations.
  - **Decoder–decoder** (self–attention): each token can only attend to previous tokens, not future ones.
  - **Encoder–decoder**: the decoder attends to the encoder output to find relevant information from the input sequence.
- Other operations (position–wise):
  - **Feed forward layers**
  - **Residual connections**: help gradients flow through deep networks
  - **Layer norm**: stabilizes training
- **Positional encoding**: position information is added to the embeddings

1.2.2 Encoder–encoder attention



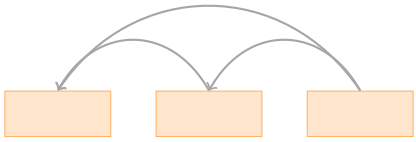
Encoder Self-Attention

The one we have discussed earlier:

- Multi–head
- Scaled dot–product
- Self–attention: Q, K, V are computed from the same input matrix X

Update embeddings of tokens from the input sequence

1.2.3 Decoder–decoder attention



Masked Decoder Self-Attention

During training, the decoder must not look at future tokens. This is achieved by masking the attention matrix:

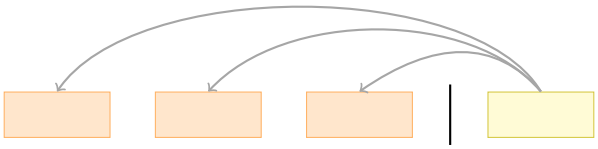
$$W'_{ij} \longleftarrow -\infty \quad \text{if } j > i$$

Setting future positions to $-\infty$ before softmax makes their attention weights effectively zero:

$$W = \text{softmax}(W')$$

Each token can only attend to previous tokens in the generated sequence.

1.2.4 Encoder–decoder attention



Encoder-Decoder Attention

This is not self-attention — Q, K, V come from different sources:

- Q — from the decoder (what the decoder is looking for)
- K, V — from the encoder (what the input sequence contains)

$$Q \in \mathbb{R}^{L_D \times d}, \qquad K \in \mathbb{R}^{L_E \times d}, \qquad V \in \mathbb{R}^{L_E \times d}$$

where L_D — decoder sequence length, L_E — encoder sequence length. The decoder attends to the most relevant tokens from the input sequence to generate the next output token.

1.2.5 Other operations

All other operations in the Transformer are applied independently to each token (position-wise) — unlike attention, they do not involve interactions between tokens.

- **Feed forward layers** — a small two-layer network applied to each token separately:

$$FFN(x) = \max(0,\ xW_1 + b_1)W_2 + b_2 \rightarrow GeLU$$

- **Residual connections** — the input is added to the output of each sub-layer:

$$\text{output} = F(x) + x$$

- **Layer norm** — normalizes the values within each token.

1.2.6 Positional encoding

Attention does not know the order of tokens — "cat ate mouse" and "mouse ate cat" look the same to it. To fix this, a positional vector is added to each token embedding before entering the model:

Attention and position–wise feed forward layers look at the input sequence as a bag of tokens	$\Rightarrow$	We need to encode positional information into the input embeddings

Variant from the initial paper:

$$PE_{(\text{pos},2i)} = \sin(\text{pos}\,/10000^{2i/d_{\text{model}}})$$

$$PE_{(\text{pos},2i+1)} = \cos(\text{pos}\,/10000^{2i/d_{\text{model}}})$$

where pos is the position of the token in the sequence and i is the dimension index. Even dimensions use sin, odd dimensions use cos. This gives each position a unique vector that the model can learn to interpret.

Another common approach — **trainable vectors**

1.2.7 Transformer size

The size of a Transformer is defined by four hyperparameters:

- Number of layers (blocks) in encoder/decoder
- Hidden dimensionality (d_model)
- Number of heads
- Dimensionality of inner feed–forward layers (usually 4 × hidden dimensionality)

Standard model; 6+6 layers, H=512, 8 heads, 65M params

1.2.8 Training details

- Loss — standard cross-entropy (NLL), lower is better:

$$NLL(y_{1:M}) = -\sum_{t=1}^{M} \log p(y_t|y_{t-1})$$

  Instead perplexity is usually reported — higher is better:

$$\text{Perplexity}(y_{1:M}) = 2^{\frac{1}{M} NLL(y_{1:M})}$$

- Teacher forcing for decoder
- Adam optimizer: adaptive learning rate for each parameter.
- Learning rate schedule with warm–up: lr grows linearly for the first *warmup* steps, then decays:

$$lr = d_{\text{model}}^{-0.5} \cdot \min(\text{step-num}^{-0.5}, \text{step-num} \cdot \text{warmup}^{-1.5})$$

1.2.9 Training tricks

- Label smoothing $y_{ls} = (1 - \alpha) \cdot y_{hot} + \alpha/K$
- Residual dropout — to the output of each sub–layer(before add+norm) + to the sums of the embeddings and the positional encodings
- BPE/Word–piece (shared embeddings for input/output)
- Model averaging (average last k checkpoints — SWA)

1.2.10 Output generation: greedy search

At each step the decoder assigns a probability to every word in the vocabulary.The goal is to find the sequence with the highest joint probability —not just the best word at each step, but the best sentence overall. Exhaustive search is infeasible (50000^{20} combinations for a 20-token sentence), so smarter algorithms are needed — **greedy search or beam search**.

The probability of a sequence factorizes as:

$$p(x) = p(x_0)\, p(x_1|x_0) \ldots p(x_t|x_0, \ldots, x_{t-1})$$

The goal is to maximize this product over the entire sequence.

greedy search:

$$y_t = \text{argmax}_{y \in \mathcal{Y}} \; P(y \mid y_1, \ldots, y_{t-1}, C)$$

The problem is that a locally optimal choice at each step does not guarantee a globally optimal sequence. For example:

Greedy search result	**Better sequence**
$P = 0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$	$P = 0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$

Greedy search picked a higher probability at step 2 (0.4 vs 0.3),but ended up with a worse overall sequence.

1.2.11 Output generation: beam search

Greedy search always picks the single best token at each step — this is fast but often leads to suboptimal sequences. That is why **beam search** was introduced — instead of one candidate it keeps k best at each step:

- At step choose k best candidates ($k =\sim 5$)
- Stop when k candidates with <END> token have been generated
- Choose the best one using a length-penalized score:

$$\frac{1}{L^{\alpha}} \log P(y_1, \ldots, y_L \mid C), \quad \alpha = 0.75$$

1.2.12 BLEU metric for NMT

After training we need to evaluate how good the translation is. **BLEU** compares the model output with a reference translation by counting how many n-grams appear in both sequences.

N-gram precision — what fraction of n-grams in the output appear in the reference:

$$p_n = \frac{\sum_{n\text{-gram} \in hyp} \text{count}_{clip}(n\text{-gram})}{\sum_{n\text{-gram} \in hyp} \text{count}(n\text{-gram})}$$

$$\text{count}_{clip} = \min(\text{count}, \text{max-ref-count})$$

Brevity penalty — penalizes translations that are too short:

$$\text{BP} = \begin{cases} e^{(1-|\,\text{ref}\,|/|\,\text{hyp}\,|)} & if |\,\text{ref}\,| > |\,\text{hyp}\,| \\ 1 & \text{otherwise} \end{cases}$$

Result (higher is better):

$$BLEU = BP \cdot \exp\left[\frac{1}{N}\sum_{n=1}^{N} \log p_n\right]$$

1.3 Metrics and Inference for sequence generation

1.4 Transformer–based LMs and contextualized word embeddings: ELMO, BERT, GPT

1.5 Transformer for Computer Vision